

Inteligencia Artificial



Inteligencia Artificial

Agentes de aprendizaje y machine learning

Referencias:

<https://www.youtube.com/watch?v=fM8yalhITFc>

<https://www.youtube.com/watch?v=oT3arRRB2Cw>

<https://www.youtube.com/watch?v=ijJYeLFtiVM>

<https://www.youtube.com/watch?v=8-nt3Urok4E>

https://www.youtube.com/watch?v=k964_uNn3l0&t=159s

<https://www.youtube.com/watch?v=mjbWGx6Xgvg&t=9s>

<https://www.youtube.com/watch?v=z5rmY-LV7ME>

<https://www.youtube.com/watch?v=kl6tyEi5eso>

<https://www.youtube.com/watch?v=MU3cLsSfnME&t=214s>

REINVENTEMOS
EL FUTURO



Aprendizaje Automático (Machine Learning)

En esta semana se explora el campo de los agentes de aprendizaje.

Esta es una introducción al terreno del Machine Learning, examinando cómo los agentes pueden aprender de los datos para tomar decisiones informadas y precisas.

Se revisará las técnicas de aprendizaje supervisado, un paradigma clave en el que los agentes aprenden a partir de ejemplos etiquetados. Además, se mostrará la implementación práctica de árboles de decisión. Finalmente, el trabajo sienta las bases para la comprensión de las redes neuronales, modelos computacionales inspirados en la estructura del cerebro humano, que han revolucionado el campo de la IA.

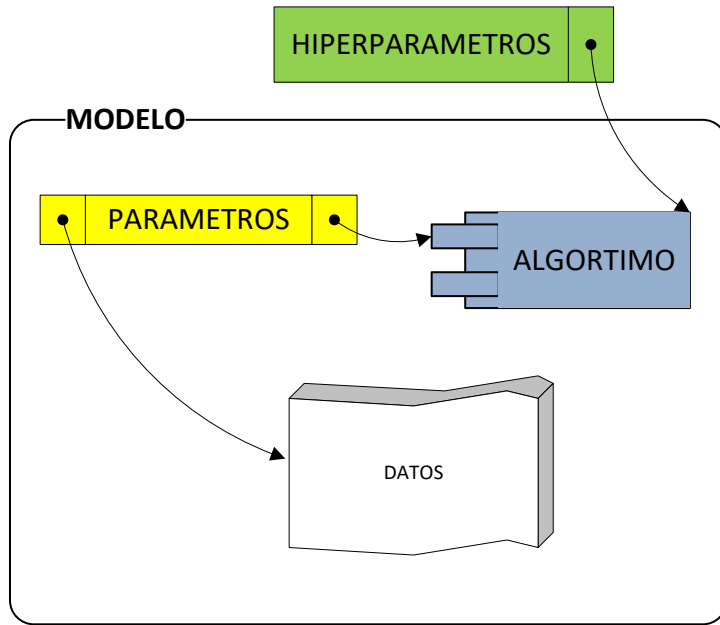
Componentes de un modelo de aprendizaje automático

El algoritmo de aprendizaje, se puede decir que es la receta o procedimiento que se desea aplicar sobre los datos.

Los **parámetros del modelo**, estos son variables internas del modelo de aprendizaje automático que se aprenden a partir de los datos durante el proceso de entrenamiento.

Los **hiperparámetros**, son valores externos al modelo que permiten configurarlo y que no se aprenden de los datos. Estos valores son seleccionados mediante algún proceso de optimización antes del entrenamiento y definen la estructura del modelo

Los datos, la calidad y el tamaño de un conjunto de datos pueden afectar el rendimiento de los sistemas de aprendizaje automático. Un conjunto de datos demasiado pequeño puede no ser representativo, un conjunto de datos demasiado grande puede ser difícil de procesar.



Tipos de técnicas de aprendizaje

Un **agente de aprendizaje** es aquel que lee observaciones sobre el mundo, las procesa, construye un modelo y lo utiliza para mejorar su rendimiento futuro. Los agentes de aprendizaje se utilizan para resolver tareas complejas cuando los diseñadores no pueden enumerar todas las posibilidades de percepción y las acciones correspondientes, y en casos en los que el agente necesita adaptarse a la entrada cambiante.

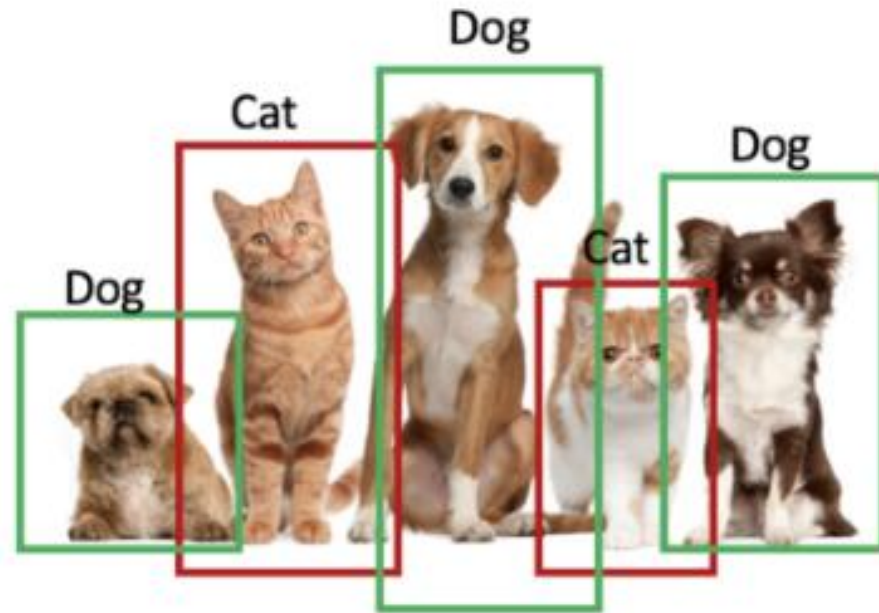
Un agente de aprendizaje permite realizar predicciones, adaptación a condiciones cambiantes. Existen tres formas de crear agentes de aprendizaje, según la retroalimentación que recibe:

- Mediante aprendizaje supervisado
- Mediante aprendizaje no supervisado
- Mediante aprendizaje por refuerzo

Introducción a las técnicas de aprendizaje supervisado

En este tipo de aprendizaje, un agente o modelo lee datos, “etiquetados”, en pares (entrada, salida). Se produce el aprendizaje y el modelo mapea la entrada a la salida.

El "etiquetado" es el proceso de asignar la respuesta correcta o la salida deseada a cada ejemplo en el conjunto de datos de entrenamiento. Estas etiquetas proporcionan al agente la información necesaria para aprender la relación entre las entradas (observaciones) y las salidas. Ejemplo: imaginemos que se quiere entrenar un agente para reconocer imágenes de gatos, entonces se necesitaría un conjunto de imágenes, y cada imagen estaría "etiquetada" como "gato" o "no gato".



Introducción a las técnicas de aprendizaje supervisado

El objetivo del aprendizaje supervisado es encontrar una función, denotada como " $f(x)$ ", que mapee las entradas (x) a las salidas (y) de la manera más precisa posible. Formalmente, se puede expresar esto como: $y = f(x)$.

x : representa la entrada o las características del ejemplo.

y : representa la salida o la etiqueta correcta.

f : es la función que el agente aproximará.

Dado un conjunto de datos que contiene puntos de datos, descritos por algunos atributos $x_1, x_2, \dots, x_n$. Cada punto de datos tiene algunos atributos, denominados " x " además de una etiqueta de clase, denominada " y " (Figura 1).

x_1	x_2	...	x_n	y

Atributos

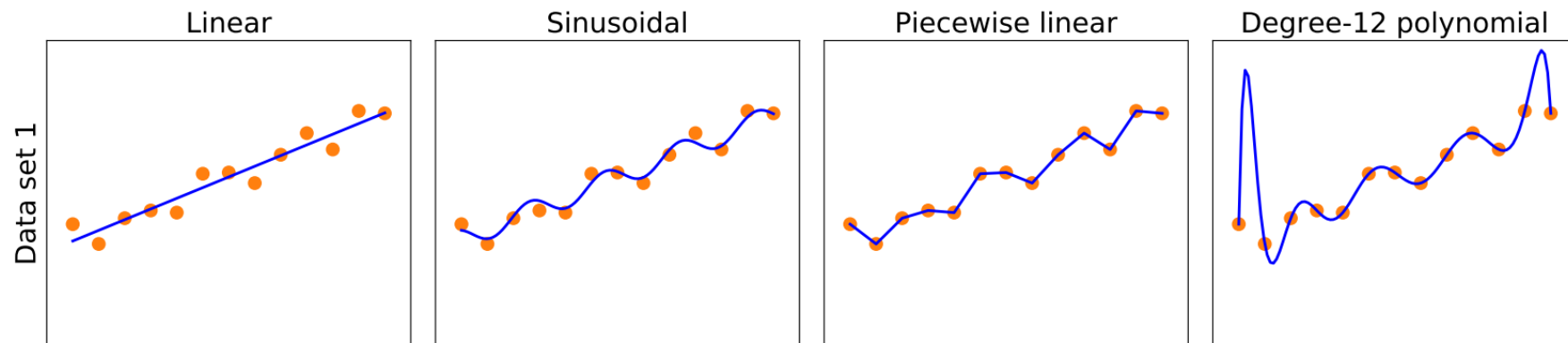
etiquetas de clase

Introducción a las técnicas de aprendizaje supervisado

El proceso de aprendizaje implica ajustar los parámetros de la función " f " para que las predicciones del agente se acerquen lo más posible a las etiquetas correctas en el conjunto de datos de entrenamiento.

Se trata entonces de buscar la función, $f()$, que pueda predecir las etiquetas de clase " y " para cualquier punto de datos dados sus atributos " x ". La función $f()$ debe mapear las entradas (x) a las salidas (y) de la manera más precisa posible. El proceso de aprendizaje implica ajustar los parámetros de la función $f()$ para que las predicciones del agente se acerquen lo más posible a las etiquetas correctas en el conjunto de datos de entrenamiento.

Esto significa que cada ejemplo en el conjunto de datos de entrenamiento está acompañado de la respuesta correcta o la salida deseada, esto permite entrenar un modelo que para que sea capaz de predecir luego valores de " y " en base a nuevos valores de " x ". Existen muchas opciones para la función $f()$, pueden ser lineales o no lineales, lineales por partes, polinomiales



Introducción a las técnicas de aprendizaje supervisado – Ejemplo

Decisión de esperar o no una mesa en un restaurante. Para este problema, la salida, **y**, es una variable booleana que se llamará WillWait; es válida para los ejemplos en los que sí se espera por una mesa. La entrada, **x**, es un vector de diez valores de atributo, se tiene un conjunto de 12 ejemplos, tomados de la experiencia de uno de los comensales. Hay $2^6 \times 3^2 \times 4^2 = 9216$ combinaciones posibles de valores para los atributos de entrada, solo se tiene la salida correcta para 12 de ellos; los otros 9204 podrían ser verdadero o falso; no se sabe. Se necesita estimar con la mayor precisión posible esos 9204 valores de salida faltantes, con solo la evidencia de los 12 ejemplos.

Example	Input Attributes										Output
	Alt	Bar	Fri	Hun	Pat	Price	Rain	Res	Type	Est	WillWait
x ₁	Yes	No	No	Yes	Some	\$\$\$	No	Yes	French	0–10	y ₁ = Yes
x ₂	Yes	No	No	Yes	Full	\$	No	No	Thai	30–60	y ₂ = No
x ₃	No	Yes	No	No	Some	\$	No	No	Burger	0–10	y ₃ = Yes
x ₄	Yes	No	Yes	Yes	Full	\$	Yes	No	Thai	10–30	y ₄ = Yes
x ₅	Yes	No	Yes	No	Full	\$\$\$	No	Yes	French	>60	y ₅ = No
x ₆	No	Yes	No	Yes	Some	\$\$	Yes	Yes	Italian	0–10	y ₆ = Yes
x ₇	No	Yes	No	No	None	\$	Yes	No	Burger	0–10	y ₇ = No
x ₈	No	No	No	Yes	Some	\$\$	Yes	Yes	Thai	0–10	y ₈ = Yes
x ₉	No	Yes	Yes	No	Full	\$	Yes	No	Burger	>60	y ₉ = No
x ₁₀	Yes	Yes	Yes	Yes	Full	\$\$\$	No	Yes	Italian	10–30	y ₁₀ = No
x ₁₁	No	No	No	No	None	\$	No	No	Thai	0–10	y ₁₁ = No
x ₁₂	Yes	Yes	Yes	Yes	Full	\$	No	No	Burger	30–60	y ₁₂ = Yes

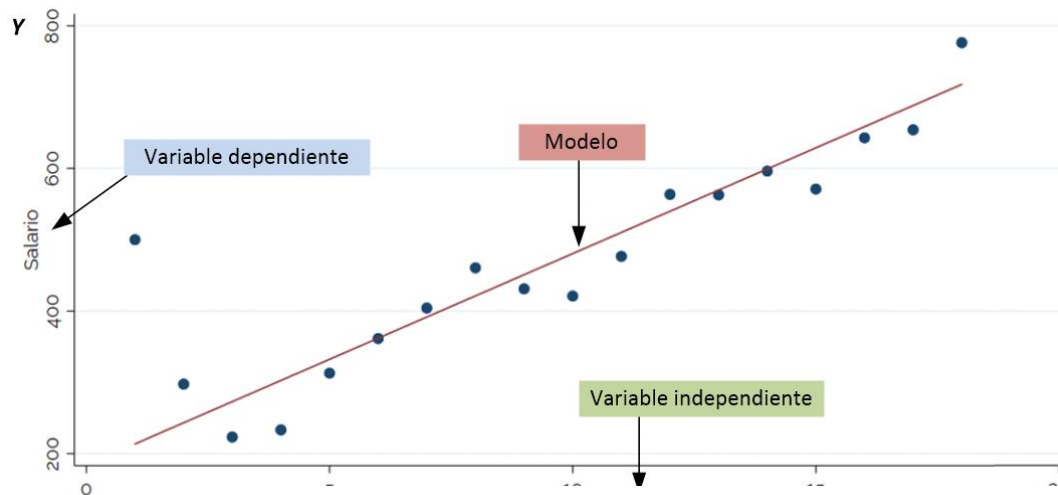
1. Alternate: si hay un restaurante alternativo adecuado cerca.
2. Bar: si el restaurante tiene una zona de bar cómoda para esperar.
3. Fri/Sat: válido los viernes y sábados.
4. Hungry: si se tiene hambre en ese momento.
5. Patrons: cuántas personas hay en el restaurante (los valores son None, Some y Full).
6. Price: rango de precios del restaurante (\$, \$\$, \$\$\$).
7. Raining: si está lloviendo afuera.
8. Reservation: si se hizo una reserva.
9. Type: el tipo de restaurante (francés, italiano, tailandés o hamburguesería).
10. WillEstimate: tiempo de espera estimado por el anfitrión: 0-10, 10-30, 30-60 o >60 minutos.

Regresión Lineal

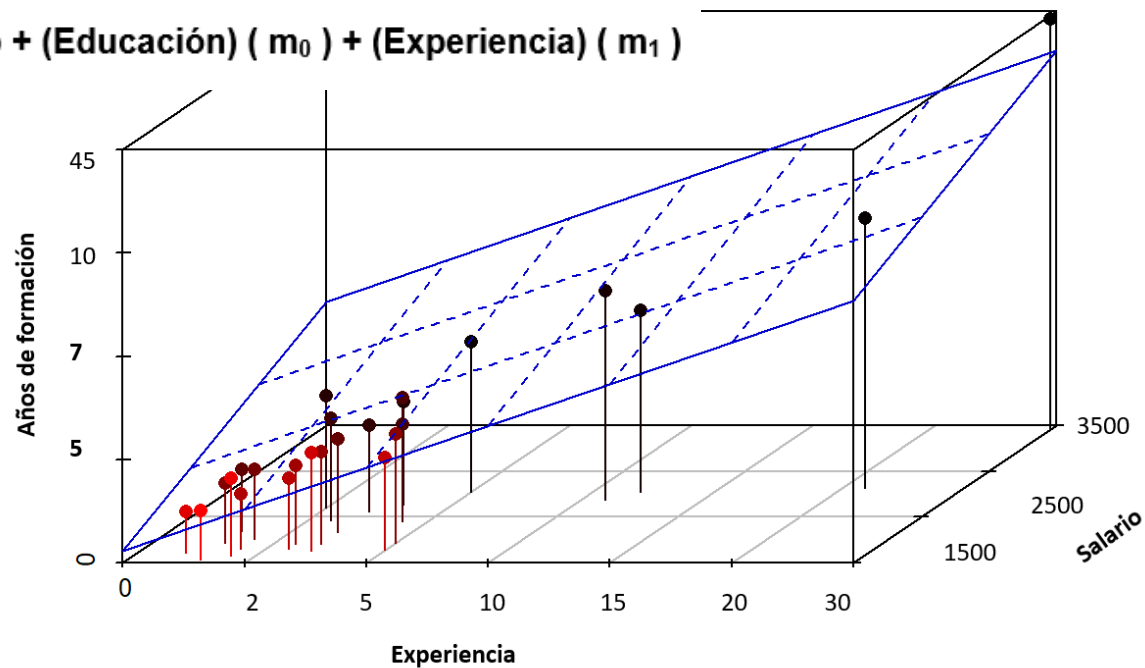
OBJETIVO: encontrar una ecuación lineal que describa de la mejor manera posible la relación entre predictores y la variable objetivo. Debe existir una relación lineal entre las variables independientes y las dependientes. Para determinar esta relación, se crea una gráfica de dispersión, una colección aleatoria de valores x , y , para ver si caen a lo largo de una línea recta.

La regresión lineal es un tipo de algoritmo de aprendizaje automático supervisado que aprende de los conjuntos de datos etiquetados y mapea los puntos de datos con las funciones lineales optimizadas, lo que permite la predicción en nuevos conjuntos de datos.

$$\text{Salario} = b + (\text{Educación}) (m_0)$$



$$\text{Salario} = b + (\text{Educación}) (m_0) + (\text{Experiencia}) (m_1)$$



Regresión Lineal

Entonces un modelo de regresión lineal en dos dimensiones es una línea recta; en tres dimensiones será un plano, y en más de tres dimensiones será un hiperplano.

Generalizando, si se tiene una relación $y = b + xm$, se trata de construir un modelo que se represente por $y' = \beta_0 + \beta_1 x_1 + \varepsilon$, donde y' es el valor a predecir, β_0 sigue siendo el intercepto también conocido como sesgo del modelo, x_1 es un atributo, variable independiente o entrada, β_1 es el peso del atributo y ε es el error que representa la diferencia entre el valor observado y el valor predicho o estimado por el modelo, este recoge el efecto de otras variables que están relacionadas con la salida pero que no se incluyeron en el modelo como variables independientes.

En términos generales la regresión lineal múltiple se puede expresar con la siguiente ecuación:

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \beta_3 x_{i3} + \dots + \beta_p x_{ip} + \varepsilon$$

Donde:

- Se tiene un número de observaciones i de datos entre 1 y n ,
- Se tiene que β_0 es el intercepto, un término constante,
- x_{ip} , representa las variables independientes,
- y_i , representa la variable dependiente,
- β_p , coeficientes de pendiente o peso para cada variable independiente,
- ε , el error del modelo, también conocido como residuo.

Regresión Lineal

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
```

- **Scikit-learn** es la librería de Python más popular para aprendizaje automático (*Machine Learning*) tradicional. Ofrece herramientas simples y eficientes para análisis de datos, clasificación, regresión y agrupamiento.
- Por otro lado, **sklearn.linear_model -> LinearRegression** es un módulo específico dentro de esa librería que sirve para crear modelos de **Regresión Lineal**, un algoritmo que predice un valor numérico continuo basándose en la relación lineal entre variables.

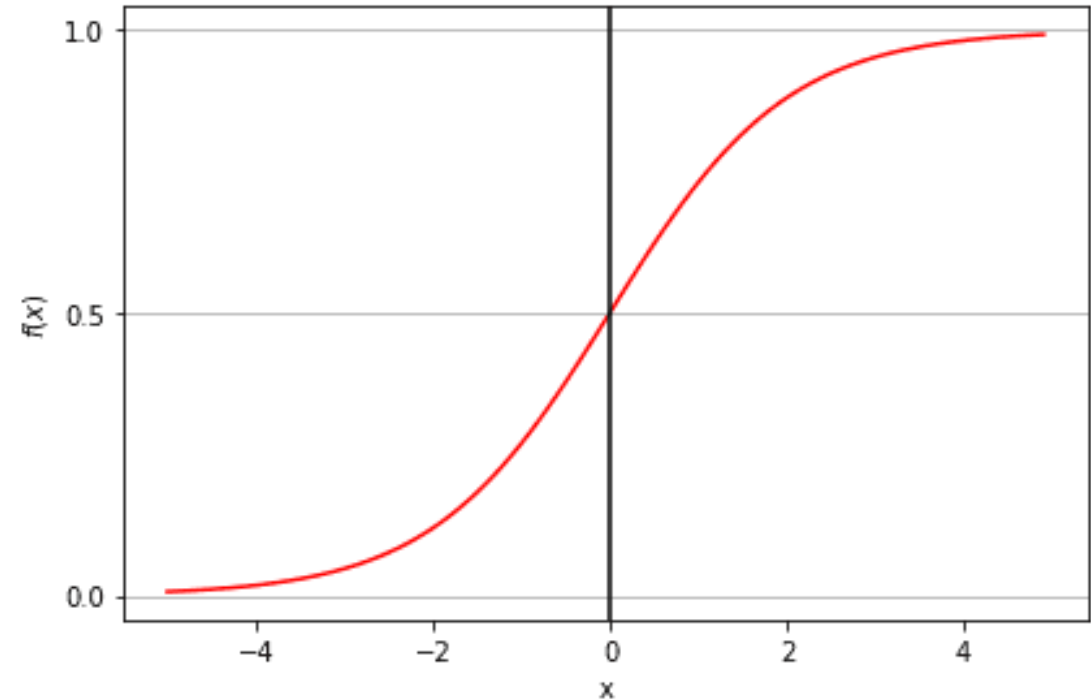
Regresión Logística

- Si la curva va en dirección del infinito positivo la predicción se convertirá en 1, y si la curva pasa al infinito negativo, la predicción se convertirá en 0. Si la salida de la función sigmoide es mayor a 0.5 y menor a 1 se dice que ese es el grado de probabilidad será de un SI o un 1, por debajo de 0.5 la probabilidad se acerca a cero, lo cual se infiere a un NO o un 0.

$$f(x) = \frac{1}{1 + e^{-x}}$$

- La regresión logística es el resultado de aplicar la función sigmoide a la regresión lineal (p representa la probabilidad en la clasificación)

$$p = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 + \dots + \beta_p x_p)}}$$



Regresión Logística

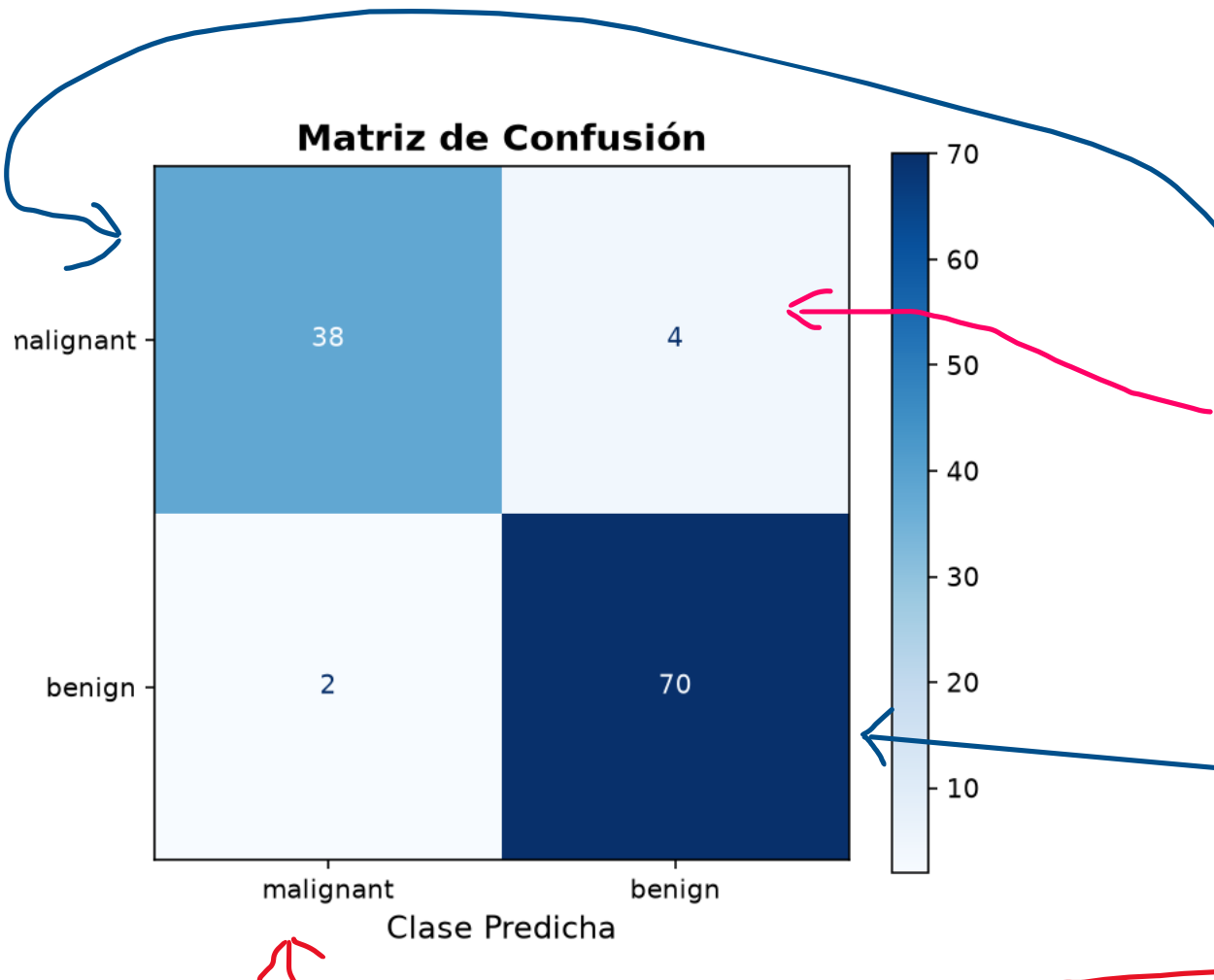
- Al hacer la aplicación de la función sigmoide, no aparece el error ε del modelo de regresión lineal, esto es porque la función no asume ningún error en la variable de salida.
- La regresión logística permitirá realizar predicciones sobre la probabilidad en una clasificación binaria.
- Para realizar un análisis de regresión logística, se puede empezar por identificar la pregunta adecuada para identificar el tipo de problema y saber si se corresponderá con la regresión logística, por ejemplo:
 - ¿Los días de lluvia afectan nuestras ventas mensuales? (sí o no)
 - ¿Qué tipo de actividad de tarjeta de crédito realiza el cliente? (autorizado, fraudulento o potencialmente fraudulento)

Regresión Logística

- `sklearn.linear_model` -> **LogisticRegression** es la herramienta de Scikit-learn diseñada para resolver problemas de **clasificación**, a pesar de que su nombre incluya la palabra "regresión". En lugar de predecir un número continuo (como el precio de una casa), predice la **probabilidad** de que un evento ocurra y clasifica los datos en categorías discretas.
- Por ejemplo, se utiliza para responder preguntas de tipo "sí o no": ¿Este correo es spam?, ¿esta transacción es un fraude?, o ¿el paciente tiene una enfermedad?

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer # Dataset de cáncer de mama
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
```

Matriz de confusión en problemas de clasificación binaria

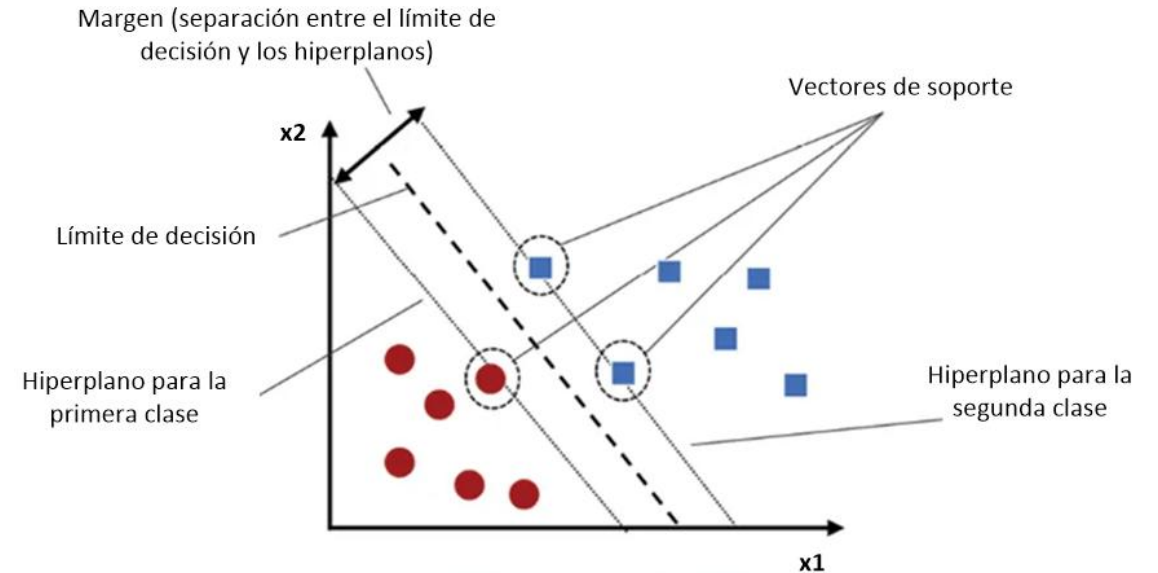


- **Verdaderos Negativos (TN):** número de personas con cáncer maligno que fueron correctamente identificadas.
- **Falsos Positivos (FP):** número de personas con cáncer benigno que fueron incorrectamente clasificadas como con cáncer maligno (Error Tipo I).
- **Falsos Negativos (FN):** número de personas con cáncer maligno que fueron incorrectamente clasificadas como con cáncer benigno (Error Tipo II).
- **Verdaderos Positivos (TP):** número de personas con cáncer benigno que fueron correctamente clasificadas.

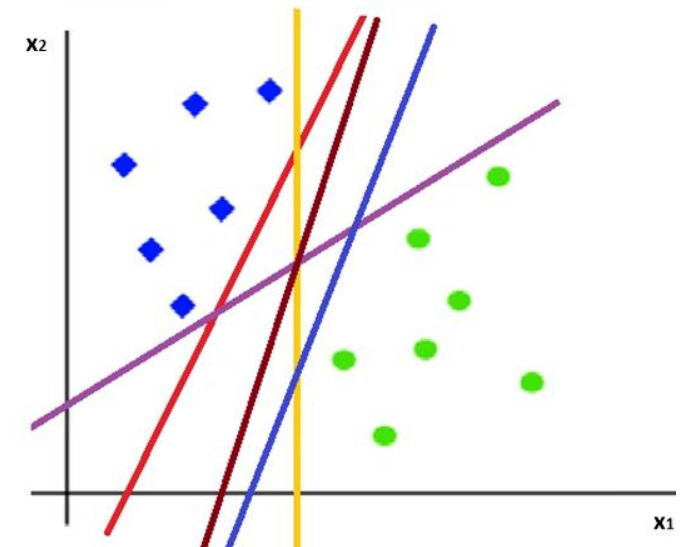
Máquinas de Soporte Vectorial (SVM): fundamentos, clasificación

OBJETIVO: Las máquinas de soporte vectorial (SVM) son un tipo de algoritmo de aprendizaje supervisado que tiene su base en la idea de “clasificador de margen máximo” que es realizar una clasificación binaria mediante un hiperplano lineal que separe los datos en dos clases con la máxima distancia posible al punto más cercano a cada clase.

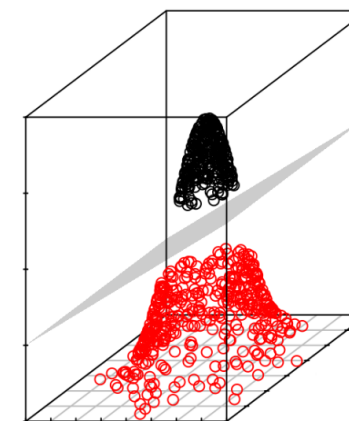
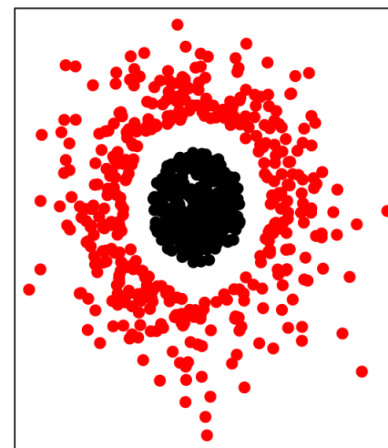
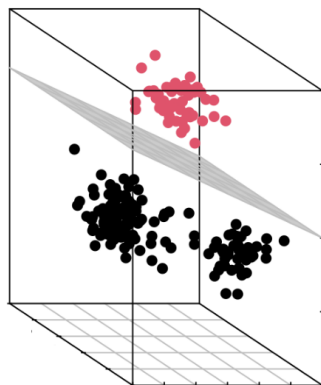
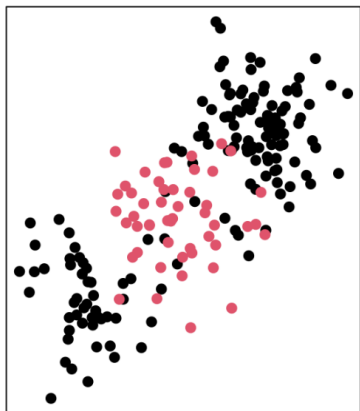
margen máximo es una referencia que se considera un límite de decisión en la clasificación que a su vez está definida por los vectores de soporte, los cuales son los puntos de datos más cercanos al hiperplano.



Hiperplano con datos linealmente separables.



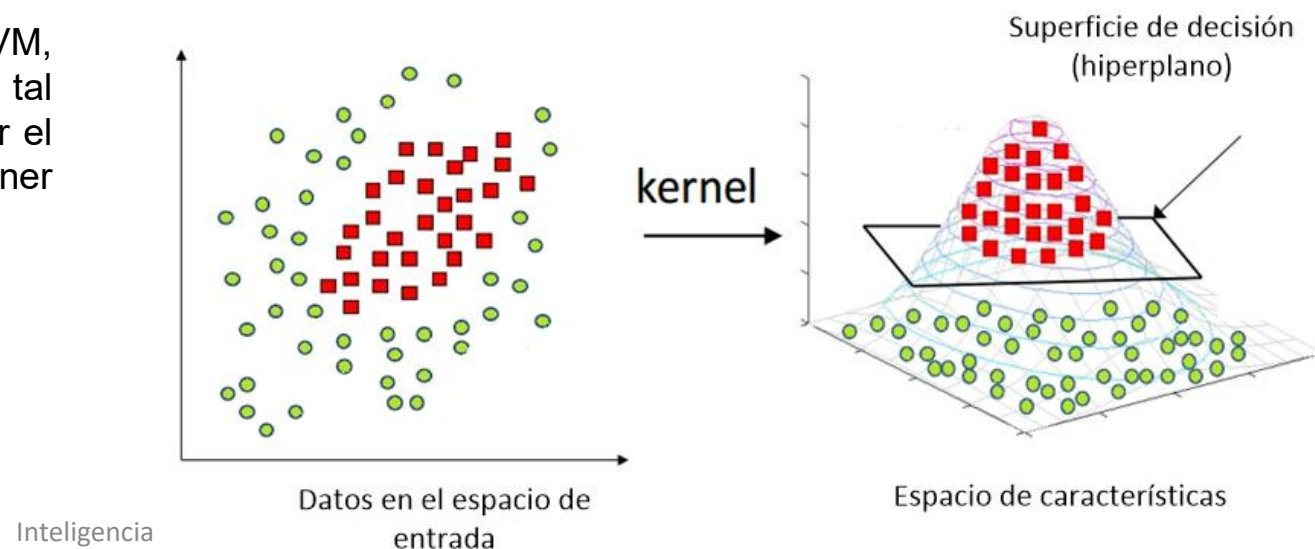
Máquinas de Soporte Vectorial (SVM): fundamentos, clasificación



El truco del Kernel le da una gran versatilidad al algoritmo SVM, consiste en aplicar una transformación a los datos de tal manera que se aumenta la dimensionalidad antes de aplicar el algoritmo de aprendizaje, de tal manera que se consigue tener un hiperplano que separa las clases.

Tipos de Kernel:

- Lineal
- Polinomial
- Radial
- Sigmoide



Máquinas de Soporte Vectorial (SVM): fundamentos, clasificación

```
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
```

StandardScaler. Es una herramienta de **preprocesamiento de datos** que se encarga de la **estandarización** o normalización de las variables numéricas.

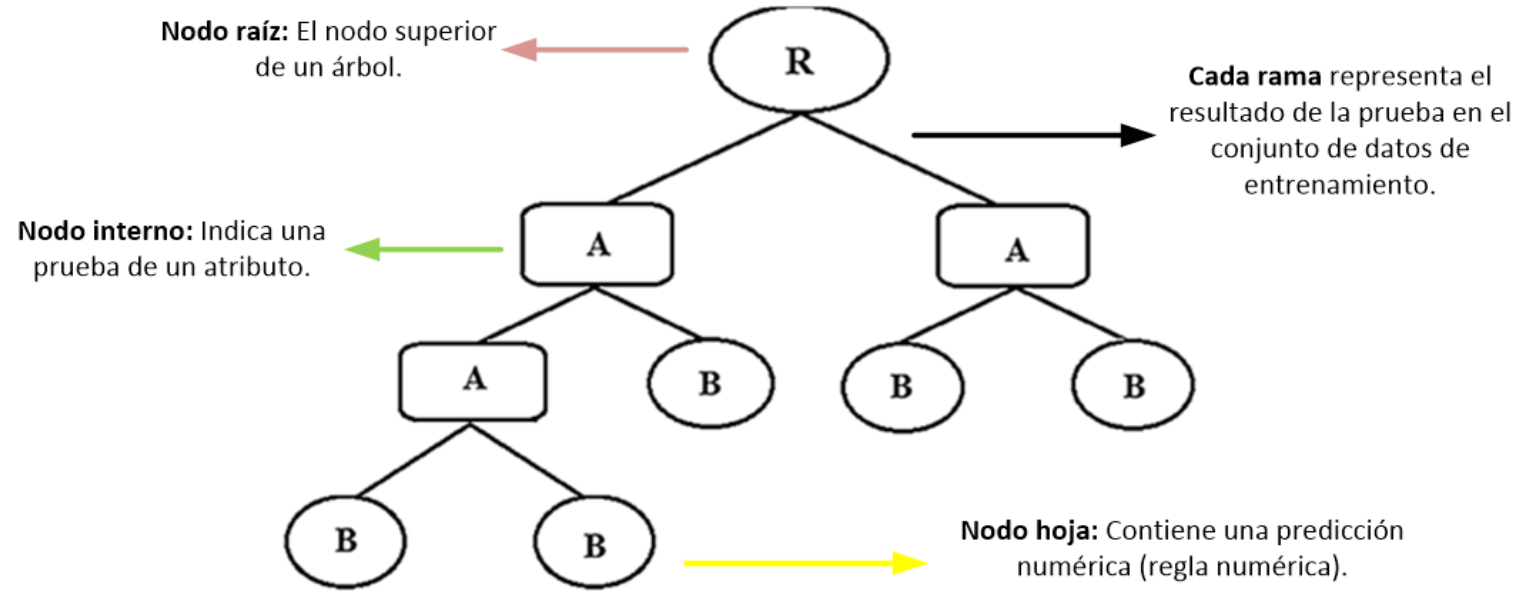
- **¿Qué hace?** Transforma los datos para que su **media sea igual a 0** y su **desviación estándar sea igual a 1** (puntuación Z).
- Los modelos de Machine Learning no entienden el contexto de los números. Si hay una columna "Edad" (valores de 0 a 100) y otra "Salario Anual" (valores de 15,000 a 200,000), el modelo pensará que el Salario es muchísimo más importante solo porque los números son más grandes.
- StandardScaler escala ambas columnas para que compartan un rango similar, evitando que las variables con números grandes distorsionen el entrenamiento.

SVC. Clasificador de Vectores de Soporte. Es un **modelo de Machine Learning** utilizado para resolver problemas de **clasificación**.

- **¿Qué hace?** Busca trazar una línea o frontera (llamada hiperplano) que separe de forma óptima dos o más categorías de datos (por ejemplo: "Correo Spam" vs. "Correo No Spam", o "Cliente que comprará" vs. "Cliente que no comprará").
- **¿Cómo funciona?** Intenta dejar el máximo espacio posible (margen) entre la línea de separación y los puntos de datos más cercanos de cada categoría. Esos puntos más cercanos se llaman "vectores de soporte".
- **¿Por qué es poderoso?** Utiliza una técnica llamada *Kernel Trick* que le permite clasificar datos complejos que no se pueden separar con una simple línea recta, proyectándolos tridimensionalmente para encontrar una solución.

Árboles de decisión

- El árbol de decisión no es un algoritmo paramétrico ya que no asume una forma funcional predeterminada. A diferencia de la regresión lineal, o el SVM, un árbol de decisión no establece una ecuación matemática global para modelar los datos.
- Lo que hace un árbol es dividir el espacio de características en una serie de regiones rectangulares disjuntas. Dentro de cada región, la predicción es una constante. La forma exacta y número de pasos no se conoce hasta que el árbol se construye a partir de los datos por eso se dice que la estructura del modelo es determinada por los datos.
- El número de "parámetros" crece con la complejidad del árbol, que a su vez se adapta a la complejidad de los datos. Esta adaptabilidad es una característica distintiva de los modelos no paramétricos.



Árboles de decisión

- Los nodos que no se dividen más se denominan nodos hoja. Cada nodo hoja representa una clase de salida para problemas de clasificación o un valor predicho para problemas de regresión.
- **¿Cómo se produce la división de los nodos?**. Los criterios de división son fundamentales para determinar qué característica y qué umbral maximizan la homogeneidad de los nodos resultantes.
- En problemas de clasificación, los criterios más comunes son la impureza de Gini y la ganancia de información (basada en la entropía). Ambas medidas son numéricamente similares. Toman valores pequeños si la mayoría de las observaciones pertenecen a la misma clase en un nodo. Por el contrario, se maximizan si el número de observaciones en todas las clases de un nodo tiende a ser igual. Un nodo con clases mixtas se denomina impuro.

Árboles de decisión

- **Impureza o índice de Gini.** Cuantifica la probabilidad de clasificación errónea de un elemento elegido aleatoriamente en el conjunto de datos. El valor mínimo del índice de **Gini es 0** y esto ocurre **cuando el nodo es puro**, lo que significa que **todos sus elementos pertenecen a una única clase**, por lo tanto, ese nodo no se volverá a dividir. La división óptima se elige por las características con menor índice de Gini.
- **Ganancia de información basada en entropía.** Es una medida de información que indica el desorden de las características con respecto al objetivo, busca identificar subconjuntos más homogéneos respecto a la variable objetivo. Similar al índice de Gini, la división óptima se elige por la característica con menor entropía. Alcanza su valor máximo cuando la probabilidad de las dos clases es la misma, y un nodo es puro cuando la entropía alcanza su valor mínimo, que es 0.
- Tanto G como H, no se emplean en problemas de regresión.

$$\left\{ \begin{array}{l} G = 1 - \sum_{i=1}^K p_i^2 \\ K, \text{ es el numero de clases o categorías} \\ p_i, \text{ proporción de elementos de clase } i \text{ en el nodo} \end{array} \right.$$

$$\left\{ \begin{array}{l} H = - \sum_{i=1}^K p_i^2 \cdot \log_2(p_i) \\ K, \text{ es el numero de clases o categorías} \\ p_i, \text{ proporción de elementos de clase } i \text{ en el nodo} \end{array} \right.$$

Árboles de decisión

- **Impureza o índice de Gini.** Cuantifica la probabilidad de clasificación errónea de un elemento elegido aleatoriamente en el conjunto de datos. El valor mínimo del índice de **Gini es 0** y esto ocurre **cuando el nodo es puro**, lo que significa que **todos sus elementos pertenecen a una única clase**, por lo tanto, ese nodo no se volverá a dividir. La división óptima se elige por las características con menor índice de Gini.
- **Ganancia de información basada en entropía.** Es una medida de información que indica el desorden de las características con respecto al objetivo, busca identificar subconjuntos más homogéneos respecto a la variable objetivo. Similar al índice de Gini, la división óptima se elige por la característica con menor entropía. Alcanza su valor máximo cuando la probabilidad de las dos clases es la misma, y un nodo es puro cuando la entropía alcanza su valor mínimo, que es 0.
- Tanto G como H, no se emplean en problemas de regresión.

$$\left\{ \begin{array}{l} G = 1 - \sum_{i=1}^K p_i^2 \\ K, \text{ es el numero de clases o categorías} \\ p_i, \text{ proporción de elementos de clase } i \text{ en el nodo} \end{array} \right.$$

$$\left\{ \begin{array}{l} H = - \sum_{i=1}^K p_i^2 \cdot \log_2(p_i) \\ K, \text{ es el numero de clases o categorías} \\ p_i, \text{ proporción de elementos de clase } i \text{ en el nodo} \end{array} \right.$$

Árboles de decisión

```
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import matplotlib.pyplot as plt
import seaborn as sns
```

seaborn (sns). Es una librería de Python especializada en la **visualización de datos estadísticos**. Se construye sobre la base de matplotlib y se utiliza para crear gráficos atractivos e informativos con muy pocas líneas de código.

- **Gráficos comunes:** Mapas de calor (correlaciones), gráficos de dispersión (scatter plots), gráficos de caja (box plots) y gráficos de distribución (histograms).

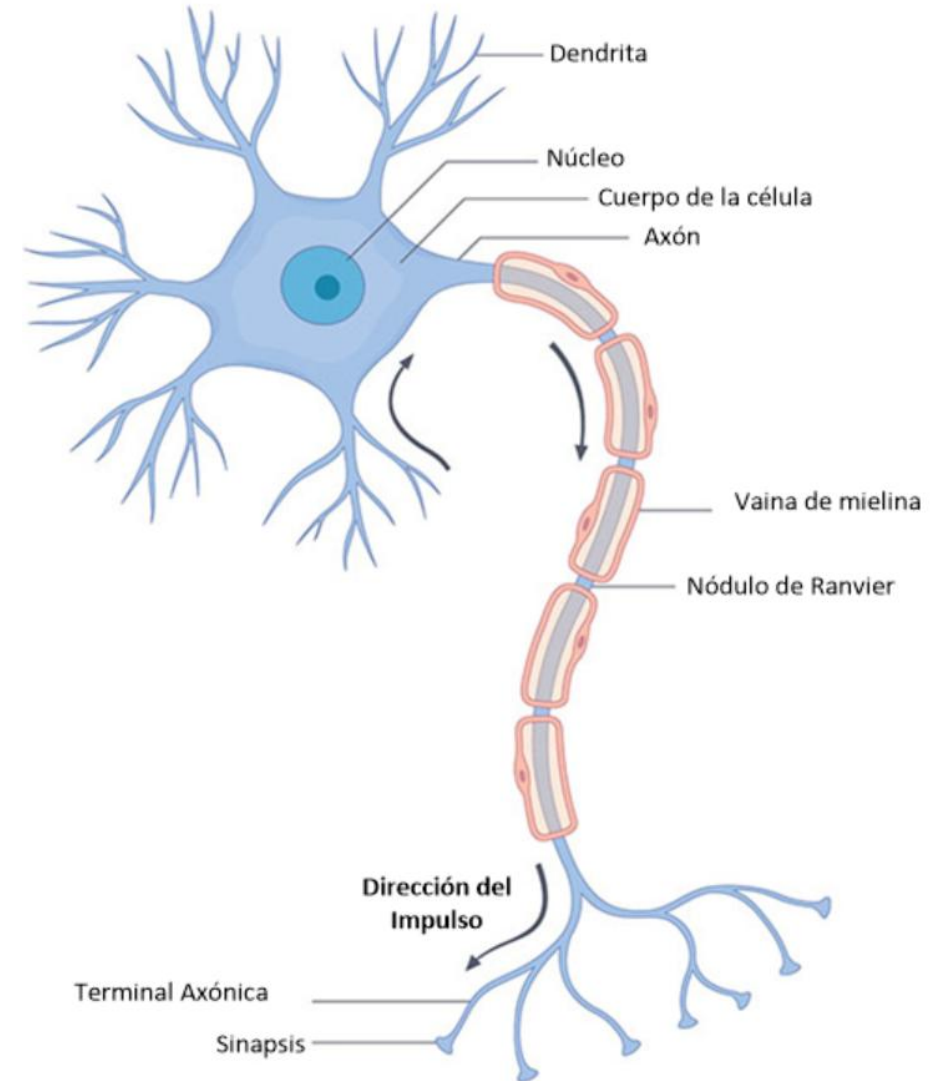
sklearn.tree.DecisionTreeClassifier. Es un **modelo de Machine Learning** para clasificación que imita la forma en que los humanos tomamos decisiones: mediante una estructura de árbol con preguntas de "sí" o "no".

- **¿Cómo funciona?:** Toma los datos y empieza a dividirlos en ramas haciendo preguntas sobre las características más importantes. Por ejemplo: *¿El cliente tiene ingresos mayores a \$50k?* Si la respuesta es sí, va por una rama; si es no, va por otra. Repite esto hasta llegar a una hoja que da la predicción final (ej. "Aprobado" o "Rechazado").
- **Ventaja principal:** Es fácil de interpretar. A diferencia de las "cajas negras" como las redes neuronales, en un árbol de decisión puedes ver exactamente qué lógica siguió el algoritmo para clasificar un dato.

sklearn.tree.plot_tree. Es la herramienta específica de Scikit-Learn que se utiliza para **dibujar y visualizar el árbol de decisión**. Convierte las reglas matemáticas del modelo DecisionTreeClassifier en un diagrama visual de nodos, flechas y condiciones. Permite ver qué preguntas está haciendo el árbol en cada nivel, cuántos datos caen en cada decisión y qué tan seguro está el modelo en cada paso.

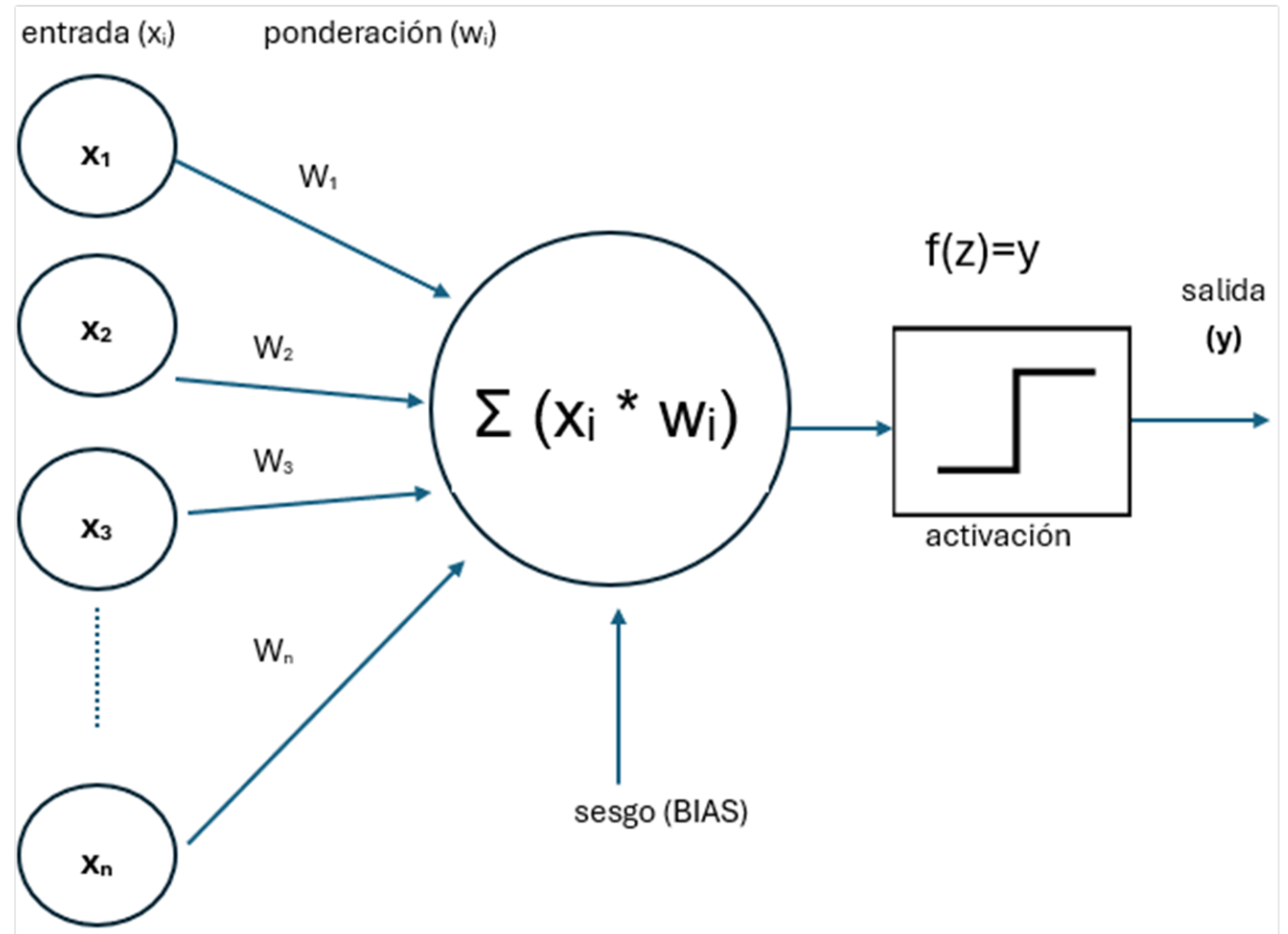
Perceptrón, redes neuronales, funciones de activación, arquitectura

- El cerebro humano contiene una red increíblemente compleja de miles de millones de células nerviosas llamadas neuronas. De manera muy simplificada, una neurona biológica recibe señales de otras neuronas a través de sus dendritas.
- Estas señales son **ponderadas** en su importancia, si la suma de las señales recibidas supera un cierto umbral, la neurona se "activa" o "dispara", enviando una señal a lo largo de su axón hacia otras neuronas, siguiendo la dirección del impulso, y conectándose mediante la sinapsis
- Cada neurona o célula nerviosa consta de tres partes fundamentales en su estructura: las dendritas, el cuerpo de la célula en el centro de las dendritas, y en su interior el núcleo. El Axón es una prolongación similar al tronco de un árbol que comunica unas neuronas con otras y forma los nervios que conectan con el resto del organismo.



Perceptrón, redes neuronales, funciones de activación, arquitectura

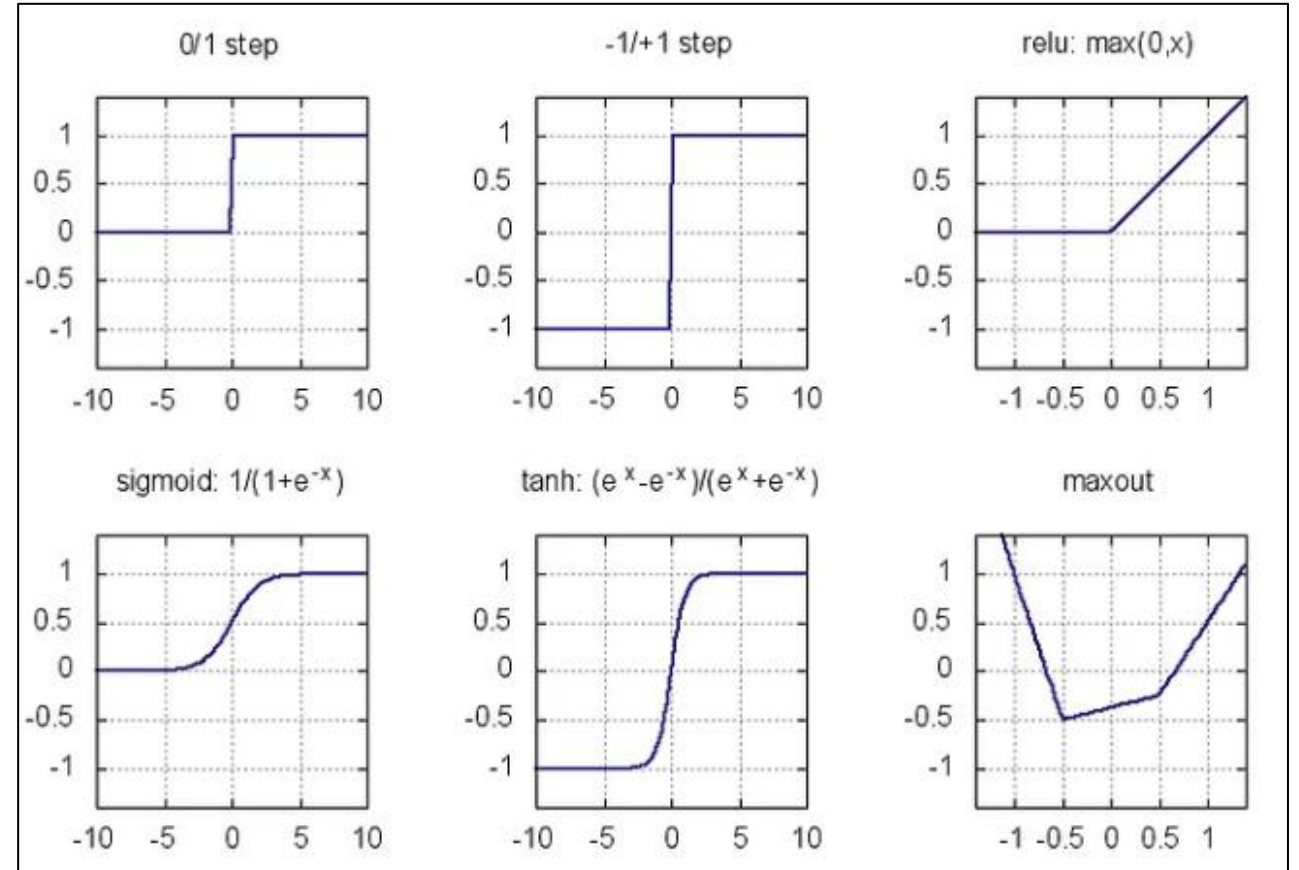
- Una neurona, recibe la información sensorial del mundo exterior, envía señales a diferentes partes del organismo, transforma y transmite las señales eléctricas. Este entendimiento es la inspiración biológica para el primer modelo de una neurona artificial.
- Una red neuronal es un programa o modelo de aprendizaje automático que toma decisiones de forma similar al cerebro humano, al emplear procesos que imitan la forma en que las neuronas biológicas trabajan juntas para identificar fenómenos, sopesar opciones y llegar a conclusiones.
- Para comprender la complejidad de una red neuronal profunda o multicapa, se debe empezar por el bloque de construcción fundamental denominado Perceptrón un algoritmo de clasificación binaria que representa una matematización inicial de la idea de una neurona biológica.



Perceptrón, redes neuronales, funciones de activación, arquitectura

¿Qué Significa que se "active"? que una neurona se "active" significa que la salida de su función de activación es significativamente diferente de cero. En otras palabras, la neurona está transmitiendo una señal que indica que ha detectado una característica relevante en sus entradas. Hay dos tipos de activación:

- Activación fuerte: una salida cercana al valor máximo de la función de activación (por ejemplo, cercano a 1 en una función sigmoide) indica una fuerte activación. La neurona está "segura" de que ha detectado la característica que está diseñada para reconocer.
- Activación débil: una salida cercana a cero (o al valor mínimo, según la función) indica una activación débil. La neurona no está segura de haber detectado la característica, o la característica no está presente en las entradas.

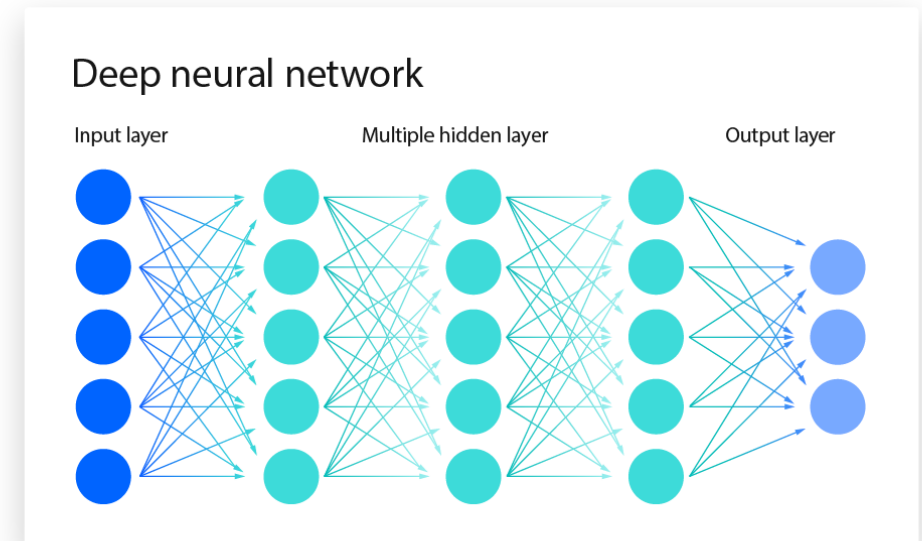


Perceptrón, redes neuronales, funciones de activación, arquitectura

Una red neuronal multicapa **MLP**, consiste en:

- **Una capa de entrada (Input layer).** Aquí no se realizan cálculos, simplemente reciben las características de entrada. El número de nodos en esta capa es igual al número de características del conjunto de datos o dataset.
- **Una o más capas ocultas (Multiple hidden layers).** Estas son las capas que se encuentran entre la entrada y la salida. Cada neurona en una capa oculta es un Perceptrón, o una versión modificada del mismo, que recibe como entrada las salidas de todas las neuronas de la capa anterior. La presencia de estas capas es lo que permite al MLP aprender relaciones no lineales.
- **Una Capa de Salida (Output layer).** Produce el resultado final de la red. El número de neuronas y la función de activación en esta capa dependen del tipo de problema, clasificación binaria, multiclase, o regresión.

El Teorema de Aproximación Universal (TAU)



Perceptrón, redes neuronales, funciones de activación, arquitectura

```
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
```

Para crear y entrenar una **Red Neuronal Artificial** básica para tareas de clasificación se usa el *Multi-Layer Perceptron Classifier* (Clasificador de Perceptrón Multicapa) **MLPClassifier**. A diferencia de los árboles de decisión o los modelos lineales, este algoritmo procesa la información de forma similar a como lo hace el cerebro humano, conectando "neuronas" matemáticas en diferentes capas para aprender patrones extremadamente complejos.

Funciona mediante capas ordenadas de forma secuencial:

- **Capa de Entrada (Input Layer):** Recibe los datos o características originales (por ejemplo, el tamaño, color y peso de una fruta).
- **Capas Ocultas (Hidden Layers):** Son capas intermedias de neuronas que procesan los datos. Multiplican las entradas por "pesos" numéricos y aplican funciones matemáticas de activación. Aquí es donde ocurre la "magia" y el aprendizaje profundo.
- **Capa de Salida (Output Layer):** Entrega la predicción final o la categoría asignada (por ejemplo: "Es una manzana").

Es sensible al escalado, al igual que el modelo SVC que vimos antes, las redes neuronales necesitan obligatoriamente que se preprocese los datos con `StandardScaler`. Si los datos no están en la misma escala, la red neuronal no podrá aprender correctamente.



Fin de la presentación